

[← Go Back](#)

Library

Recents viewed

ArduinoBLE

ArduinoMqttClient

WiFiNINA

Arduino SigFox for
MKRFox1200

autowp-mcp2515

Arduino_MKRENV

Arduino_MKRGPS

Arduino_APDS9960

Arduino_MKRIoTCarrier

[Home](#) / [Programming](#) / [Library](#) / **ArduinoBLE**

ON THIS PAGE

COMMUNICATION

ArduinoBLE

 **ARDUINO**  GNU Lesser General Public License v2.1 V1.4.1  Arduino  24/07/2025 Arduino <info@arduino.cc> <https://www.arduino.cc/en/Re...>  info@arduino.cc

Enables Bluetooth® Low Energy connectivity on the Arduino MKR WiFi 1010, Arduino UNO WiFi Rev2, Arduino Nano 33 IoT, Arduino Nano 33 BLE, Nicla Sense ME and UNO R4 WiFi.

This library supports creating a Bluetooth® Low Energy peripheral & central mode.

GO TO REPOSITORY

Usage/Examples

Compatibility

Releases

This library supports all the Arduino boards that have the hardware enabled for Bluetooth® Low Energy and Bluetooth® 4.0 and above; these include Nano 33 BLE, Arduino NANO 33 IoT, Uno WiFi Rev2, MKR WiFi 1010, Nicla Sense ME.

To use this library

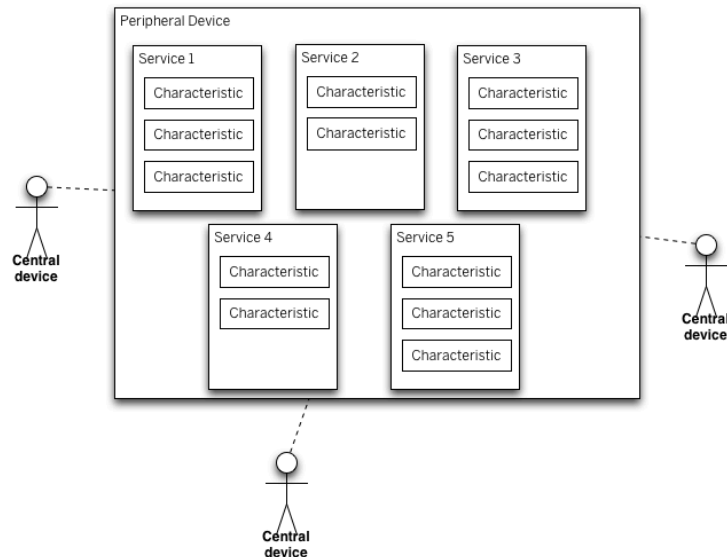
```
#include <ArduinoBLE.h>
```

A quick introduction to BLE

Bluetooth® 4.0 includes both traditional Bluetooth®, now labeled "Bluetooth® Classic", and the Bluetooth® Low Energy. Bluetooth® Low Energy is optimized for low power use at low data rates, and was designed to operate from simple lithium coin cell batteries.

Unlike standard Bluetooth® communication basically based on an asynchronous serial connection (UART) a Bluetooth® LE radio acts like a community bulletin board. The computers that connect to it are like community members that read the bulletin board. Each radio acts as either the bulletin board or the reader. If your radio is a bulletin board (called a peripheral device in Bluetooth® LE parlance) it posts data for all radios in the community to read. If your radio is a reader (called a central device in Bluetooth LE terms) it reads from any of the bulletin boards (peripheral devices) that have information about which it cares. You can also think of peripheral devices as the servers in a client-server transaction, because they contain the information that reader radios ask for. Similarly, central devices are the clients of the Bluetooth® LE world because they read information available from the peripherals.

[Help](#)



Think of a Bluetooth® LE peripheral device as a bulletin board and central devices as viewers of the board. Central devices view the services, get the data, then move on. Each transaction is quick (a few milliseconds), so multiple central devices can get data from one peripheral.

The information presented by a peripheral is structured as **services**, each of which is subdivided into **characteristics**. You can think of services as the notices on a bulletin board, and characteristics as the individual paragraphs of those notices. If you're a peripheral device, you just update each service characteristic when it needs updating and don't worry about whether the central devices read them or not. If you're a central device, you connect to the peripheral then read the boxes you want. If a given characteristic is readable and writable, then the peripheral and central can both change it.

Notify

The Bluetooth® LE specification includes a mechanism known as **notify** that lets you know when data's changed. When notify on a characteristic is enabled and the sender writes to it, the new value is automatically sent to the receiver, without the receiver explicitly issuing a read command. This is commonly used for streaming data such as accelerometer or other sensor readings. There's a variation on this specification called **indicate** which works similarly, but in the indicate specification, the reader sends an acknowledgment of the pushed data.

The client-server structure of Bluetooth® LE, combined with the notify characteristic, is generally called a **publish-and-subscribe model**.

Update a characteristic

Your peripheral should update characteristics when there's a significant change to them. For example, when a switch changes from off to on, update its characteristic. When an analog sensor changes by a significant amount, update its characteristic.

Just as with writing to a characteristic, you could update your characteristics on a regular interval. but this wastes processing power and

Central and Peripheral Devices

Central devices are **clients**. They read and write data from peripheral devices. **Peripheral** devices are **servers**. They provide data from sensors as readable characteristics, and provide read/writable characteristics to control actuators like motors, lights, and so forth.

Services, characteristics, and UUIDs

A Bluetooth® Low Energy peripheral will provide **services**, which in turn provide **characteristics**. You can define your own services, or use standard services (see section 3.4 in the [Assigned Numbers document](#)).

Services are identified by unique numbers known as UUIDs. You know about UUIDs from other contexts. Standard services have a 16-bit UUID and custom services have a 128-bit UUID. The ability to define services and characteristics depends on the radio you're using and its firmware.

Service design patterns

A characteristic value can be up to 512 bytes long. This is a key constraint in designing services. Given this limit, you should consider how best to store data about your sensors and actuators most effectively for your application. The simplest design pattern is to store one sensor or actuator value per characteristic, in ASCII encoded values.

Characteristic	Value
Accelerometer X	200
Accelerometer Y	134
Accelerometer Z	150

This is also the most expensive in memory terms, and would take the longest to read. But it's the simplest for development and debugging.

You could also combine readings into a single characteristic, when a given sensor or actuator has multiple values associated with it.

Characteristic	Value
Motor Speed, Direction	150,1
Accelerometer X, Y, Z	200,133,150

This is more efficient, but you need to be careful not to exceed the 512-byte limit. The accelerometer characteristic above, for example, takes 11 bytes as an ASCII-encoded string.

Read/write/notify/indicate

There are 4 things a central device can do with a characteristic:

Read: ask the peripheral to send back the current value of the characteristic. Often used for characteristics that don't change very

Write: modify the value of the characteristic. Often used for things that are like commands, for example telling the peripheral to turn a motor on or off.

Indicate and **Notify:** ask the peripheral to continuously send updated values of the characteristic, without the central having to constantly ask for it.

Advertising and GAP

BLE devices let other devices know that they exist by advertising using the **General Advertising Profile (GAP)**. Advertising packets can contain a device name, some other information, and also a list of the services it provides.

Advertising packets have a limited size. You will only be able to fit a single 128-bit service UUID in the packet. Make sure the device name is not too long, or you won't even be able to fit that.

You can provide additional services that are not advertised. Central devices will learn about these through the connection/bonding process. Non-advertised services cannot be used to discover devices, though. Sometimes this is not an issue. For example, you may have a custom peripheral device with a custom service, but in your central device app you may know that it also provides the Battery Service and other services.

GATT

The Bluetooth LE protocol operates on multiple layers. **General Attribute Profile (GATT)** is the layer that defines services and characteristics and enables read/write/notify/indicate operations on them. When reading more about GATT, you may encounter GATT concepts of a "server" and "client". These don't always correspond to central and peripherals. In most cases, though, the peripheral is the GATT server (since it provides the services and characteristics), while the central is the GATT client.

Library structure

As the library enables multiple types of functionality, there are a number of different classes.

`BLE` used to enable the Bluetooth® Low Energy module.

`BLEDevice` used to get information about the devices connected or discovered while scanning.

`BLEService` used to enable the services board provides or interact with services a remote board provides.

`BLECharacteristic` used to enable the characteristics board offers in a service or interact with characteristics a remote board provides.

`BLEDescriptor` used to describe a characteristic the board offers.

Used to enable the characteristics board offers in a service or interact with characteristics a remote board provides.

▼ BLECharacteristic()

Create a new Bluetooth® Low Energy characteristic.

Syntax

```
1 BLECharacteristic(uuid, properties, valueSize)
2 BLECharacteristic(uuid, properties, valueSize, fixedLength)
3 BLECharacteristic(uuid, properties, stringValue)
4
5 BLEBoolCharacteristic(uuid, properties)
6 BLEBooleanCharacteristic(uuid, properties)
7 BLECharCharacteristic(uuid, properties)
8 BLEUnsignedCharCharacteristic(uuid, properties)
9 BLEByteCharacteristic(uuid, properties)
10 BLEShortCharacteristic(uuid, properties)
11 BLEUnsignedShortCharacteristic(uuid, properties)
12 BLEWordCharacteristic(uuid, properties)
13 BLEIntCharacteristic(uuid, properties)
14 BLEUnsignedIntCharacteristic(uuid, properties)
15 BLELongCharacteristic(uuid, properties)
16 BLEUnsignedLongCharacteristic(uuid, properties)
17 BLEFloatCharacteristic(uuid, properties)
18 BLEDoubleCharacteristic(uuid, properties)
```

Parameters

uuid: 16-bit or 128-bit UUID in **String** format

properties: mask of the properties (BLEBroadcast, BLERead, BLEWriteWithoutResponse, BLEWrite, BLENotify, BLEIndicate)

valueSize: (maximum) size of characteristic value

fixedLength: if true, size of characteristic value is fixed

stringValue: value as a string

Returns

New **BLECharacteristic** with the specified **UUID** and value

Example

```
1 // Bluetooth® Low Energy Battery Level Characteristic
2 BLEUnsignedCharCharacteristic batteryLevelChar("2E10C446-46FD-4E90-BF5A-690905AF4923",
3         BLERead | BLENotify); // remote clients will be notified
```

▼ bleCharacteristic.uuid()

Query the UUID of the specified BLECharacteristic

Syntax

```
1 bleCharacteristic.uuid()
```

Parameters

None

Returns

UUID of the Bluetooth® Low Energy service as a **String**.

Example

```
1 // Bluetooth® Low Energy LED Switch Characteristic
2 BLEByteCharacteristic switchCharacteristic("19B106");
3
4
5 Serial.print("Switch characteristic UUID = ");
6 Serial.println(switchCharacteristic.uuid());
```

▼ bleCharacteristic.properties()

Query the property mask of the specified BLECharacteristic.

Syntax

```
1 bleCharacteristic.properties()
```

Parameters

None

Returns

Properties of the characteristic masked (BLEBroadcast, BLERead, BLEWriteWithoutResponse, BLEWrite, BLENotify, BLEIndicate)

Example

```
1 // Bluetooth® Low Energy LED Switch Characteristic
2 BLEByteCharacteristic switchCharacteristic("19B106");
3
4
5 byte properties = switchCharacteristic.properties;
6
7 if (properties & BLERead) {
8     // characteristic is readable ...
9 }
10
11 if (properties & (BLEWrite | BLEWriteWithoutResponse)) {
12     // characteristic is writable ...
13 }
```

▼ bleCharacteristic.valueSize()

Query the maximum value size of the specified BLECharacteristic.

Syntax

```
1 bleCharacteristic.valueSize()
```

Parameters

None

Returns

The **maximum value** size of the characteristic (in bytes)

Example

```
1 // Bluetooth® Low Energy LED Switch Characteristic
2 BLEByteCharacteristic switchCharacteristic("19B106");
3
4
5
6 Serial.print("value size = ");
7 Serial.println(switchCharacteristic.valueSize());
```

▼ bleCharacteristic.value()

Query the current value of the specified BLECharacteristic.

Syntax

```
1 bleCharacteristic.value()
```

Parameters

None

Returns

The **current value** of the characteristic, value type depends on the constructor used

Example

```
1 // Bluetooth® Low Energy LED Switch Characteristic
2 BLEByteCharacteristic switchCharacteristic("19B10
3
4
5
6 if (switchCharacteristic.value()) { // any val
7   Serial.println("LED on");
8   digitalWrite(ledPin, HIGH); // will turn the
9 } else { // a 0 value
10   Serial.println(F("LED off"));
11   digitalWrite(ledPin, LOW); // will turn the
12 }
```

▼ bleCharacteristic.valueLength()

Query the current value size of the specified BLECharacteristic.

Syntax

```
1 bleCharacteristic.valueLength()
```

Parameters

None

Returns

The **current value** size of the characteristic (in bytes)

Example


```
1 // Bluetooth® Low Energy LED Switch Characteristic
2 BLEByteCharacteristic switchCharacteristic("19B106");
3
4
5
6 Serial.print("value length = ");
7 Serial.println(switchCharacteristic.valueLength());
```

▼ bleCharacteristic.readValue()

Read the current value of the characteristic. If the characteristic is on a remote device, a read request will be sent.

Syntax

```
1 bleCharacteristic.readValue(buffer, length)
2 bleCharacteristic.readValue(value)
```

Parameters

buffer: byte array to read value into length: size of buffer argument in bytes

value: variable to read value into (by reference)

Returns

Number of bytes read

Example

```
1 while (peripheral.connected()) {
2     // while the peripheral is connected
3
4     // check if the value of the simple key characteristic has been updated
5     if (simpleKeyCharacteristic.valueUpdated()) {
6         // yes, get the value, characteristic is 1 byte long
7         byte value = 0;
8
9         simpleKeyCharacteristic.readValue(value);
10
11         if (value & 0x01) {
12             // first bit corresponds to the right button
13             Serial.println("Right button pressed");
14         }
15
16         if (value & 0x02) {
17             // second bit corresponds to the left button
18             Serial.println("Left button pressed");
19         }
20     }
```

▼ **bleCharacteristic.writeValue()**

Write the value of the characteristic. If the characteristic is on a remote device, a write request or command will be sent.

Syntax

```
1 bleCharacteristic.writeValue(buffer, length)
2 bleCharacteristic.writeValue(value)
```

Parameters

buffer: byte array to write value with

length: number of bytes of the buffer argument to write

value: value to write

Returns

1 on success,

0 on failure

Example

```
1 // read the button pin
2 int buttonState = digitalRead(buttonPin);
3
4 if (oldButtonState != buttonState) {
5     // button changed
6     oldButtonState = buttonState;
7
8     if (buttonState) {
9         Serial.println("button pressed");
10
11         // button is pressed, write 0x01 to turn
12         ledCharacteristic.writeValue((byte)0x01)
13     } else {
14         Serial.println("button released");
15
16         // button is released, write 0x00 to turn
17         ledCharacteristic.writeValue((byte)0x00)
18     }
19 }
```

▼ **bleCharacteristic.setEventHandler()**

Set the event handler (callback) function that will be called when the specified event occurs.

Syntax

```
1 bleCharacteristic.setEventHandler(eventType, callback)
```

Parameters

eventType: event type (BLESubscribed, BLEUnsubscribed, BLERead, BLEWritten)

callback: function to call when the event occurs

Returns

Nothing

Example

```
1 // create switch characteristic and allow remote
2 BLEByteCharacteristic switchCharacteristic("19B10
3
4
5
6
7 // assign event handlers for characteristic
8 switchCharacteristic.setEventHandler(BLEWritten,
9
10
11
12 void switchCharacteristicWritten(BLEDevice central,
13 // central wrote new value to characteristic, u
14 Serial.print("Characteristic event, written: ")
15
16 if (switchCharacteristic.value()) {
17   Serial.println("LED on");
18   digitalWrite(ledPin, HIGH);
19 } else {
20   Serial.println("LED off");
21   digitalWrite(ledPin, LOW);
22 }
23 }
```

▼ bleCharacteristic.broadcast()

Broadcast the characteristics value as service data when advertising.

Syntax

```
1 bleCharacteristic.broadcast()
```

Parameters

None

Returns

1 on success,
0 on failure

Example

```
1 // create button characteristic and allow remote c
2 BLEByteCharacteristic buttonCharacteristic("19B106
3
4
5
6 buttonCharacteristic.broadcast();
```

▼ bleCharacteristic.written()

Query if the characteristic value has been written by another Bluetooth® Low Energy device.

Syntax

```
1 bleCharacteristic.written()
```

Parameters

None

Returns

true if the characteristic value has been written by another Bluetooth® Low Energy device,
false otherwise

Example

```

1 // Bluetooth® Low Energy LED Switch Characteristic
2 BLEByteCharacteristic switchCharacteristic("19B"
3
4
5
6
7 // listen for Bluetooth® Low Energy peripheral:
8 BLEDevice central = BLE.central();
9
10 // if a central is connected to peripheral:
11 if (central) {
12   Serial.print("Connected to central: ");
13   // print the central's MAC address:
14   Serial.println(central.address());
15
16   // while the central is still connected to peripheral:
17   while (central.connected()) {
18     // if the remote device wrote to the characteristic:
19     // use the value to control the LED:
20     if (switchCharacteristic.written()) {
21       if (switchCharacteristic.value()) { // if true, LED is on
22         Serial.println("LED on");
23         digitalWrite(ledPin, HIGH);      // turn the LED on (HIGH is the voltage level)
24       } else {
25         Serial.println(F("LED off"));
26         digitalWrite(ledPin, LOW);       // turn the LED off by making the pin LOW
27       }
28     }
29   }
30 }

```

▼ bleCharacteristic.subscribed()

Query if the characteristic has been subscribed to by another Bluetooth® Low Energy device.

Syntax

```
1 bleCharacteristic.subscribed()
```

Parameters

None

Returns

true if the characteristic value has been subscribed to by another Bluetooth® Low Energy device,

false otherwise

Example

```

1 // Bluetooth® Low Energy Battery Level Characteristic
2 BLEUnsignedCharCharacteristic batteryLevelChar("2E",
3         BLERead | BLENotify); // remote clients will
4
5
6
7
8
9 if (batteryLevelChar.subscribed()) {
10     // set a new value , that well be pushed to
11     batteryLevelChar.writeValue(0xab);
12 }

```

▼ bleCharacteristic.addDescriptor()

Add a BLEDescriptor to the characteristic.

Syntax

```
1 bleCharacteristic.addDescriptor(bleDescriptor)
```

Parameters

bleDescriptor: descriptor to add to the characteristic

Returns

Nothing

Example

```

1 // Bluetooth® Low Energy Battery Level Characteristic
2 BLEUnsignedCharCharacteristic batteryLevelChar("2E",
3         BLERead | BLENotify); // remote clients will
4
5 BLEDescriptor batteryLevelDescriptor("2901", "min
6
7
8
9
10
11 batteryLevelChar.addDescriptor(batteryLevelDes

```

▼ bleCharacteristic.descriptorCount()

Query the number of Bluetooth® Low Energy descriptors discovered for the characteristic

Syntax

```
1 bleCharacteristic.descriptorCount()
```

Parameters

None

Returns

The **number of Bluetooth® Low Energy descriptors** discovered for the characteristic

Example

```
1 // loop the descriptors of the characteristic and
2   for (int i = 0; i < characteristic.descriptorCount(); i++)
3       BLEDescriptor descriptor = characteristic.getDescriptor(i);
4
5       // ...
6   }
```

▼ bleCharacteristic.hasDescriptor()

Check if a characteristic has a particular descriptor.

Syntax

```
1 bleCharacteristic.hasDescriptor(uuid)
2 bleCharacteristic.hasDescriptor(uuid, index)
```

Parameters

index: index of descriptor

uuid: uuid (as a **String**)

Returns

true, if the characteristic has a matching descriptor,
otherwise **false**.

Example

```
1 if (characteristic.hasDescriptor("2901")) {  
2   Serial.println("characteristic has description"  
3 }
```

▼ bleCharacteristic.descriptor()

Get a BLEDescriptor that represents a characteristics Bluetooth® Low Energy descriptor.

Syntax

```
1 bleCharacteristic.descriptor(index)  
2 bleCharacteristic.descriptor(uuid)  
3 bleCharacteristic.descriptor(uuid, index)
```

Parameters

index: index of descriptor

uuid: uuid (as a **String**)

Returns

BLEDescriptor that represents a characteristics Bluetooth® Low Energy descriptor

Example

```
1 if (characteristic.hasDescriptor("2901")) {  
2   Serial.println("characteristic has description"  
3 }
```

▼ bleCharacteristic.canRead()

Query if a Bluetooth® Low Energy characteristic is readable.

Syntax

```
1 bleCharacteristic.canRead()
```

▼ bleCharacteristic.canWrite()

Query if a Bluetooth® Low Energy characteristic is writable.

None

Returns

true, if characteristic is readable,
false otherwise

Example

```
1 if (characteristic.canRead("2901")) {  
2     Serial.println("characteristic is readable");  
3 }
```

read

Perform a read request for the characteristic.

Syntax

```
1 bleCharacteristic.read()
```

Parameters

None

Returns

true, if successful,
false on failure

Example

```
1 if (characteristic.read()) {  
2     Serial.println("characteristic value read");  
3  
4     // ...  
5 } else {  
6     Serial.println("error reading characteristic");  
7 }
```

▼ **bleCharacteristic.canWrite()**

Query if a Bluetooth® Low Energy characteristic is writable.

Syntax

```
1 bleCharacteristic.canWrite()
```

Parameters

None

Returns

true, if characteristic is writable,
false otherwise

Example

```
1 if (characteristic.canWrite()) {  
2   Serial.println("characteristic is writable");  
3 }
```

▼ bleCharacteristic.canSubscribe()

Query if a Bluetooth® Low Energy characteristic is subscribable.

Syntax

```
1 bleCharacteristic.canSubscribe()
```

Parameters

None

Returns

true, if characteristic is subscribable,
false otherwise

Example

```
1 if (characteristic.canSubscribe()) {  
2     Serial.println("characteristic is subscribable")  
3 }
```

▼ `bleCharacteristic.subscribe()`

Subscribe to a Bluetooth® Low Energy characteristics notification or indications.

Syntax

```
1 bleCharacteristic.subscribe()
```

Parameters

None

Returns

true, on success,
false on failure

Example

```
1 // ...  
2  
3 // retrieve the simple key characteristic  
4 BLECharacteristic simpleKeyCharacteristic = peripheral->getCharacteristic(SIMPLE_KEY_CHARACTERISTIC_UUID);  
5  
6 // subscribe to the simple key characteristic  
7 Serial.println("Subscribing to simple key characteristic");  
8 if (!simpleKeyCharacteristic) {  
9     Serial.println("no simple key characteristic found");  
10    peripheral->disconnect();  
11    return;  
12 } else if (!simpleKeyCharacteristic.canSubscribe()) {  
13     Serial.println("simple key characteristic is not subscribable");  
14     peripheral->disconnect();  
15     return;  
16 } else if (!simpleKeyCharacteristic.subscribe()) {  
17     Serial.println("subscription failed!");  
18     peripheral->disconnect();  
19     return;  
20 }  
21  
22 // ...
```

Query if a Bluetooth® Low Energy characteristic is unsubscribable.

Syntax

```
1 bleCharacteristic.canUnsubscribe()
```

Parameters

None

Returns

true, if characteristic is unsubscribable,
false otherwise

Example

```
1 if (characteristic.canUnsubscribe()) {  
2   Serial.println("characteristic is unsubscribable")  
3 }
```

▼ bleCharacteristic.unsubscribe()

Unsubscribe to a Bluetooth® Low Energy characteristics notifications or indications.

Syntax

```
1 bleCharacteristic.unsubscribe()
```

Parameters

None

Returns

true, on success,
false on failure

Example

```
1 // ...
2
3 // retrieve the simple key characteristic
4 BLECharacteristic simpleKeyCharacteristic = peripheral->getCharacteristic("Simple Key Characteristic");
5
6 // subscribe to the simple key characteristic
7 Serial.println("Subscribing to simple key characteristic");
8 if (!simpleKeyCharacteristic) {
9     Serial.println("no simple key characteristic found");
10    peripheral.disconnect();
11    return;
12 } else if (!simpleKeyCharacteristic.canSubscribe()) {
13     Serial.println("simple key characteristic is not subscribable");
14    peripheral.disconnect();
15    return;
16 } else if (!simpleKeyCharacteristic.subscribe(callback)) {
17     Serial.println("subscription failed!");
18    peripheral.disconnect();
19    return;
20 }
21
22 // ...
23
24 simpleKeyCharacteristic.unsubscribe();
```

▼ bleCharacteristic.valueUpdated()

Has the characteristics value been updated via a notification or indication.

Syntax

```
1 bleCharacteristic.valueUpdated()
```

Parameters

None

Returns

true, if the characteristics value been updated via a notification or indication

Example

```
1 while (peripheral.connected()) {
2     // while the peripheral is connected
3
4     // check if the value of the simple key character
5     if (simpleKeyCharacteristic.valueUpdated()) {
6         // yes, get the value, characteristic is 1
7         byte value = 0;
8
9         simpleKeyCharacteristic.readValue(value);
10
11         if (value & 0x01) {
12             // first bit corresponds to the right button
13             Serial.println("Right button pressed");
14         }
15
16         if (value & 0x02) {
17             // second bit corresponds to the left button
18             Serial.println("Left button pressed");
19         }
20     }
21 }
```

Was this article helpful?



Connect and Contribute

Project Hub
GitHub Repository
Forum

Product Compliance
Help Center
Trademarks & Licensing